Evidence Binding and Deferred Decisions in Graph-Assisted Error Diagnosis

A Controlled Case Study of a Gemma-Based Developer Workflow

Hwan Kim
hwankim1103@gmail.com

Abstract

Structured evidence and intermediate review do not by themselves establish the correctness of a remediation plan. We examine a Gemma workflow combining RDF observations, SHACL validation, rule-generated candidates, and conditional remediation. Two controlled comparisons cover three author-built programs, twelve contexts, and 120 evaluation units. LLM review deferred 3/18 negative-context units, all repetitions of one context; each subsequent first plan again contained conditional COPY. With matched raw inputs and output contracts, candidate-free and candidate-provided conditions produced 36/36 and 33/36 valid final responses. Seventeen candidate-provided outputs selected incorrect insertion lines matching source-evidence identifier numbers. These observations motivate separate evaluation of evidence binding, contextual decisions, and final plans; their mechanisms and generalization remain unverified.

1 Introduction and related work

An `ENOENT` log identifies a missing path but does not distinguish a wrong reference, omitted deployment asset, or unavailable external mount. A useful diagnostic workflow must connect logs, source locations, and deployment observations while retaining uncertainty about the running environment.

RCAgent integrates diagnostic tools with LLM reasoning [5]; AetherLog augments log analysis with knowledge-graph retrieval [1]. RepoGraph represents repository code relationships for software agents [3]. SWE-agent studies agent–computer interfaces [10], whereas Agentless uses a constrained sequence of localization, repair, and validation [9]. Our system is a restricted diagnostic and planning workflow, without autonomous repository-wide repair. Unlike ChatRepair's conversational use of test feedback [8], diagnosis here does not execute or iteratively revise changes.

We ask whether LLM review changes candidate acceptance (E1), whether additional rule candidates alter evidence binding under a common input/output contract (E2), and whether eligible plans restore supported failures. The contribution is a controlled case study separating decisions, contract violations, and execution outcomes, with concrete examples of location errors and deferred decisions followed by conditional plans. Identifier-number correspondence and fallback behavior were analyzed after observing the final results, not as preregistered causal hypotheses.

2 Diagnostic workflow

Rule-based extraction assigns EV identifiers to logs and SC identifiers to source evidence. An SC identifier indexes evidence; it is not a file's line number. The RDF graph connects `ReadFailure`, `StackFrame`, and `SourceRead` observations when execution scope, target path, and source location agree. Optional archive observations record file presence, Docker COPY coverage, and ignore policy. A packaging candidate requires a matching read candidate, an archived file, a supported COPY omission, and an allowing ignore policy under matching scope, paths, and archive identity.

Preprint.

SHACL checks declared structure and evidence references [7] before and after fixed SPARQL rules construct candidates [6]. These checks neither prove a root cause nor inspect the running image. The evaluated inference is graph-pattern rule application, not OWL entailment. Gemma receives candidate fields and raw context, not an RDF traversal interface. It judges acceptance or deferral, selects an allowed plan, and supplies explanatory text. A server binds accepted plans to templates containing preconditions, changes, verification, and rollback instructions.

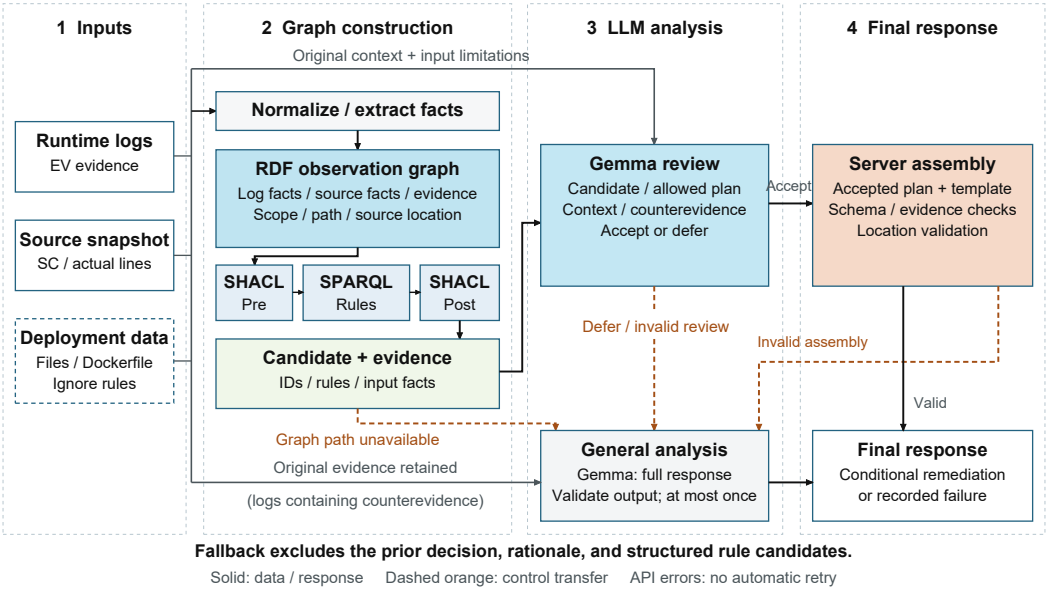


Figure 1: Workflow and fallback boundary. Solid arrows carry data or responses; dashed arrows trigger routing. Fallback retains original evidence, including logs containing counterevidence, but excludes the previous decision, rationale, and structured candidates. Remediation execution is a separate evaluation.

Unsupported graph inputs use general analysis. After review deferral or output-validation failure, one general-analysis call reuses the original context (Figure 1). In the final experiments, this input does not include the prior decision or rationale. Raw counterevidence remains available. API failures do not trigger retries.

3 Controlled evaluation

Cases and reference policy. Three author-built Node.js programs implement a shipping-fee CLI, an HTTP template server, and a settlement batch. Each has one base COPY-omission failure. Four authored contexts are combined with collected logs and source, producing twelve program–context pairs: P_s: approved static asset and matching build, accept; P_u: unknown supply/build information, permit conditional acceptance; N_e: explicit external-volume supply, defer; N_b: supplied/observed build mismatch, defer. The policy permits a plan contingent on unresolved information but defers a candidate contradicted by an explicit supply contract or build mismatch. For example, a batch observation uses `supply_kind=external_volume` and `repository_file_role=example_only`; the unknown context instead uses `unknown` and `unconfirmed`. These injected observations and author-defined references are not independently adjudicated operational ground truth. All contexts still produce rule candidates.

Comparisons and controls. E1 compares automatic acceptance R_{auto} (one unit per context, twelve total) with Gemma review R_{llm} (three repetitions, 36 units). Both use candidate-bound server assembly. E2 compares no added rule candidates B_{eq} with added candidates G_{eq}, each with 36 units. Both models select target, source location, evidence IDs, and COPY insertion line. Actual requests match in all 36 pairs except `rule_candidates`; prompts, schemas, and generation settings also match. Both inputs retain raw failure-selection signals. Candidates provide neither complete

patches nor the insertion-line answer. E1 and E2 are separate comparisons with different binding arrangements.

The API model was `gemma-4-31b-it`: temperature 0, `thinkingLevel: minimal`, 4,096 output tokens maximum, requested seeds 42/43/44, 180-second timeout, and 60-second call spacing. Prompts instructed the model in Korean and required Korean natural-language explanations; code, paths, and identifiers retained their original notation. Quoted explanations in this paper are English translations. Seeds do not ensure provider determinism. Candidate packets were prepared offline. There were 108 primary attempts and twenty fallback attempts; one timeout is retained without replacement. Earlier contract failures informed prompt clarification on these same development contexts. This is neither a held-out evaluation nor an independent replication. Three repetitions do not create three independent faults.

Outcomes. A positive acceptance requires a valid primary accept decision and reference-matching bindings. A negative acceptance is a valid accept where the reference requires defer. Explicit deferral, contract blockage, and API failure are distinct. Contract checks concern output schema, evidence, and locations, not SHACL validation or semantic correctness. Explanatory adequacy is not automatically scored.

Only P_s is execution-eligible. A fixed applicator applies the first final plan in a reproducible Docker environment; recovery requires functional, regression, and protected-file hash checks. Reference and deliberately incorrect repairs were prechecked. This follows execution-based assessment exemplified by SWE-bench [2], while limiting claims to the tests actually run [4]. Invalid outputs and API failures remain in the scheduled denominator. Identical original/normalized patches reuse execution rather than count twice.

4 Results and concrete cases

Table 1: Primary decisions and final outcomes. Pos/Neg denote authored reference contexts. Block/API are primary failures; final validity includes fallback. Recovery uses scheduled supported-context units. Different repetition counts are retained.

Arm	Pos accept	Neg accept	Neg defer	Block / API	Final valid	Recovery
R_{auto}	6/6	6/6	0/6	0 / 0	12/12	3/3
R_{llm}	18/18	15/18	3/18	0 / 0	36/36	9/9
B_{eq}	18/18	18/18	0/18	0 / 0	36/36	9/9
G_{eq}	9/18	9/18	0/18	17 / 1	33/36	6/9

4.1 Candidate fields and location selection

B_{eq} passed all 36 primary contracts but also accepted all eighteen negative-context units (Table 1). G_{eq} produced eighteen valid accepts, seventeen contract blockages, one timeout, and no explicit deferrals. Its lower negative acceptance count reflects nine blockages, not correct contextual rejection.

All seventeen blockages involved `packaging.insert_after_line`: eleven HTTP outputs selected 11 instead of actual line 6; six batch outputs selected 12 instead of line 7. These values match `SC000011` and `SC000012`. In the HTTP supported-context pair at seed 42, the shared source contains:

```
SC000011 | Dockerfile | line 6:
COPY --chown=node:node routes ./routes
```

The added packaging candidate cites Dockerfile evidence `SC000006` through `SC000013`, targets `/app/views/index.html`, and identifies `views/index.html`; it contains no insertion-line field. Selected original response fields are:

```
B_eq: "decision": "accept", "insert_after_line": 6
G_eq: "decision": "accept", "insert_after_line": 11
```

The latter was blocked by `INVALID_PACKAGING_BINDING`. Identifier/line confusion is consistent with this pattern, but candidate semantics, numeric identifiers, input length, and layout were not

independently manipulated. The observation does not identify a causal mechanism or establish an RDF disadvantage.

4.2 Review and subsequent plans

R_{llm} accepted 18/18 positive units and deferred 3/18 negative units. All deferrals were the settlement batch's external-supply context; the other five negative contexts were accepted on every repetition (Table 2). Although the defer rationale mentioned external supply, `counter_evidence_ids` remained empty: structured negative-evidence coverage was 0/18 and precision undefined because there were no citations.

Each deferred unit's subsequent first plan contained:

```
COPY --chown=node:node config ./config
```

In the first repetition, review described COPY as inconsistent with the external-volume contract. Fallback still received that original `EV000008` observation, but not the prior decision or rationale. The later plan applied "if the configuration file is a static file that should be included at image-build time" (translated from Korean). Its presence is not proof of an unconditional recommendation, harmful execution, or a semantically inappropriate final plan. It does not establish preservation of the earlier deferral either.

The frozen COPY detector missed this directory copy because its destination lacked a trailing slash. A separate post-hoc audit found COPY in at least one final plan for every negative unit across conditions. That audit inspected snippets and archive paths, not complete deployment semantics. Original scores remain preserved; non-detection is not a safety-success metric.

Table 2: Primary states by program/context. A: valid accept, D: explicit defer, X: contract blockage, T: timeout; numbers are counts. A does not denote semantic correctness. Automatic acceptance runs once; model conditions run three times.

Program	Context	R_{auto}	R_{llm}	B_{eq}	G_{eq}
CLI	P_s	A1	A3	A3	A3
	P_u	A1	A3	A3	A3
	N_e	A1	A3	A3	A3
	N_b	A1	A3	A3	A3
HTTP	P_s	A1	A3	A3	X2, T1
	P_u	A1	A3	A3	X3
	N_e	A1	A3	A3	X3
	N_b	A1	A3	A3	X3
Batch	P_s	A1	A3	A3	A3
	P_u	A1	A3	A3	X3
	N_e	A1	D3	A3	X3
	N_b	A1	A3	A3	A3

4.3 Recovery and overhead

Supported-context recovery was 3/3 versus 9/9 in E1 and 9/9 versus 6/9 in E2. All 27 executed plans passed the specified checks. The three unexecuted G_{eq} HTTP units comprised two invalid fallback source-location references and one API timeout, not three executed repairs that failed.

Fifteen of seventeen G_{eq} fallback responses passed validation, yielding 33/36 final valid responses. Total generation attempts were 39 for R_{llm}, 36 for B_{eq}, and 53 for G_{eq}: the latter exceeded B_{eq} by 47.2%. Observed E2 input/output tokens were 134,634/10,239 without candidates and 279,128/37,510 with candidates. Usage for the timeout is missing, not zero. Totals combine candidate input and fallback overhead, excluding offline graph construction.

5 Implications, limitations, and conclusion

The observations support three evaluation practices within this workflow: audit evidence IDs against actual locations; separate contract validity from contextual acceptance; and inspect final plans after intermediate deferral. The candidate-free baseline's valid negative accepts show why format validation alone is insufficient. Conditional plans require assessment of their applicability, not just the presence of a COPY command.

Limitations include three authored base faults, development reuse, injected contexts, a single API model/configuration, no independent reference adjudication, and restricted parsing, application, and tests. Repetitions are not independent projects. Numeric-ID remapping, server-side location resolution, propagation of defer rationales, and evaluation on unseen projects remain unperformed follow-ups. No statistical significance, equivalence, broad generalization, necessity of LLM review, or superiority of RDF/SHACL is claimed. Our contribution is the separation and documentation of observed failure patterns, not a verified explanation of their causes.

Selected paired requests, schemas, generation settings, raw responses, and source hashes are included in the accompanying supplement (`evidence/final_control_examples.json`). The included `verify_examples.py` checks the illustrated input difference, source-line mismatch, and defer/fallback sequence using saved records only. These checks cover the selected examples, not all 120 evaluation units, and do not establish semantic correctness. The supplement is not a complete rerunnable benchmark. Saved-output inspection is distinct from model re-execution.

References

[1] T. Cui, R. Fu, C. Liu, Y. Ji, W. Gu, S. Zhang, Y. Sun, and D. Pei. AetherLog: Log-based root cause analysis by integrating large language models with knowledge graphs. In *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*, pages 49–60. IEEE, 2025. doi: 10.1109/ISSRE66568.2025.00019.

[2] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

[3] S. Ouyang, W. Yu, K. Ma, Z. Xiao, Z. Zhang, M. Jia, J. Han, H. Zhang, and D. Yu. RepoGraph: Enhancing AI software engineering with repository-level code graph. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/4a4a3c197deac042461c677219efd36c-Abstract-Conference.html.

[4] E. K. Smith, E. T. Barr, C. Le Goues, and Y. Brun. Is the cure worse than the disease? overfitting in automated program repair. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering*, pages 532–543. Association for Computing Machinery, 2015. doi: 10.1145/2786805.2786825.

[5] Z. Wang, Z. Liu, Y. Zhang, A. Zhong, J. Wang, F. Yin, L. Fan, L. Wu, and Q. Wen. RCAgent: Cloud root cause analysis by autonomous agents with tool-augmented large language models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. Association for Computing Machinery, 2024. doi: 10.1145/3627673.3680016.

[6] World Wide Web Consortium. SPARQL 1.1 query language. W3c recommendation, World Wide Web Consortium, Mar. 2013. URL https://www.w3.org/TR/2013/REC-sparql11-query-20130321/.

[7] World Wide Web Consortium. Shapes constraint language (SHACL). W3c recommendation, World Wide Web Consortium, July 2017. URL https://www.w3.org/TR/2017/REC-shacl-20170720/.

[8] C. S. Xia and L. Zhang. Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 819–831. Association for Computing Machinery, 2024. doi: 10.1145/3650212.3680323.

[9] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE), 2025. doi: 10.1145/3715754. Article FSE037, 24 pages.

[10] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.